WX·RADIO

RETRO RADIO WEATHER APP

Midterm submission · FA26-SDET373103 Web App Development for Mobile · October 2026

Expo app + Back4App backend: logins and sign-ups, saved cities, weather forecasts, and push weather alerts sent from Back4App Cloud Code.

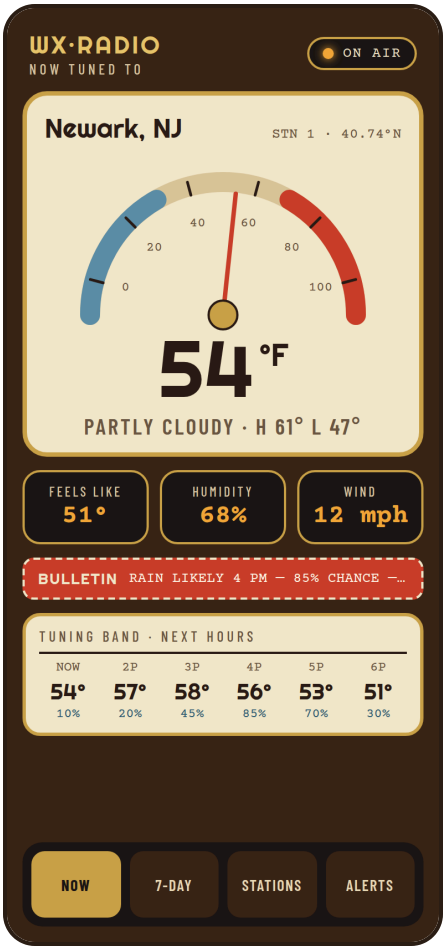
1. Design (made in Claude Design)
2. App documentation
3. API reference
4. Prompt log (what I asked Claude)

Claude Design canvas: claude.ai/artifact/H43byitsjRMZkv4VnYWy7r
Code: github.com/AjHy2003/AjHy2003.github.io → `weather-app/`

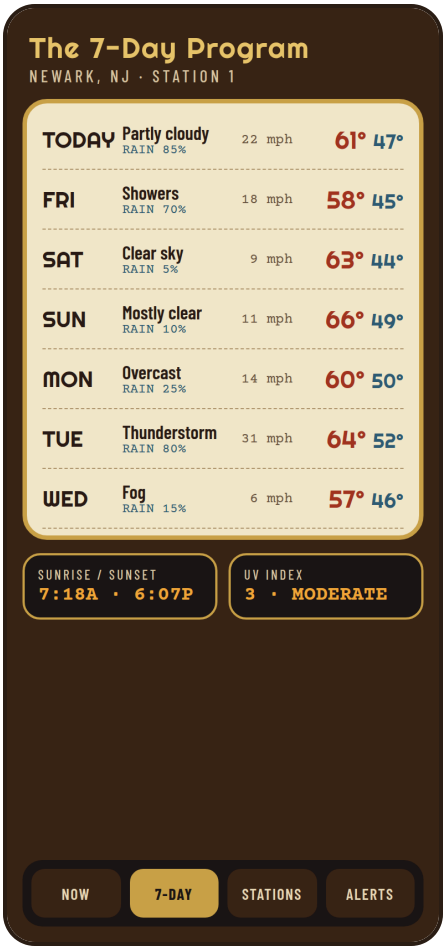
1 · Design — Claude Design

Theme: "Weather Station Radio." A 1940s tabletop radio with a walnut cabinet, cream dial face and brass trim. The temperature is an analog gauge with a needle, saved cities are radio "stations", and alerts come in as an "Emergency Broadcast" on red ticker tape.

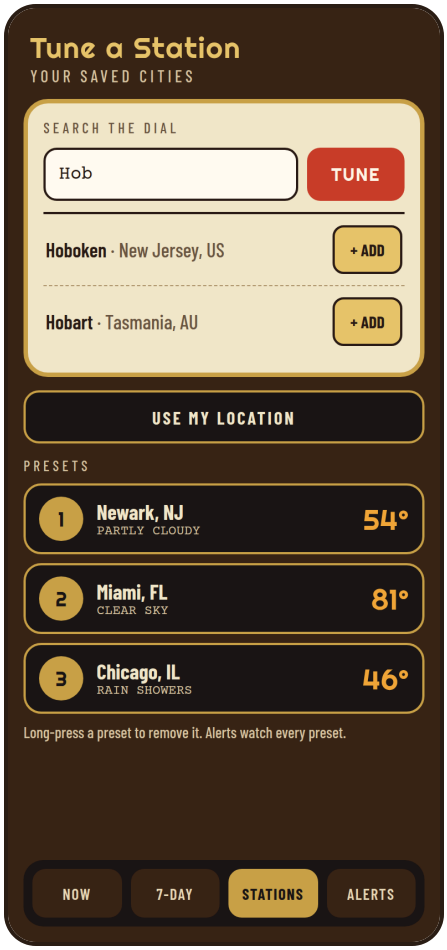
Designed in Claude Design: 5 phone screens (390 × 844) plus a style guide. The source files are in `weather-app/design/`.



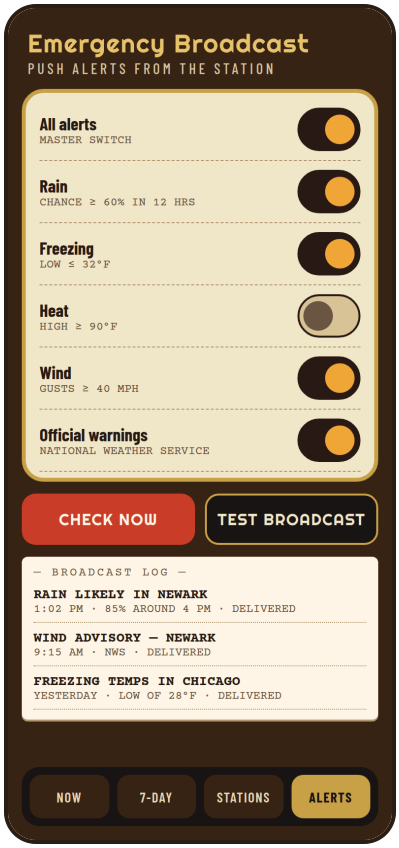
01 · NOW (HOME)



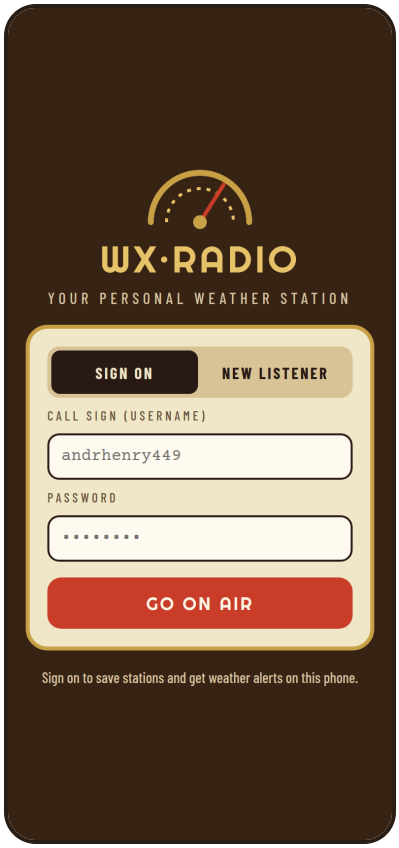
02 · 7-DAY



03 · STATIONS



04 · ALERTS + LOG



05 · SIGN ON / SIGN UP

Style guide

Weather Station Radio — style guide

A 1940s tabletop radio: walnut cabinet, cream dial face, brass trim, an analog needle and ticker-tape bulletins.

Walnut	Bakelite	Dial cream	Brass	Marquee gold	Tube amber	Signal red	Cold steel
#3b2416 · cabinet	#1c1714 · gauges, nav	#f3e6c8 · dial face	#c9a24a · trim, selected	#e9c46a · titles	#f2a93b · readouts	#c8402a · alerts, CTA	#2f5d73 · lows, rain

DISPLAY · RIGHTEOUS

54°F

Temperatures, screen titles, city names, big buttons.

LABELS · BARLOW CONDENSED

FEELS LIKE · HUMIDITY

Uppercase with wide tracking, like stamped panel labels.

READOUTS · COURIER PRIME

RAIN LIKELY 4 PM

Gauge readouts, ticker-tape bulletins, the broadcast log.

PRIMARY · GO ON AIR

SECONDARY · TEST

TAB · SELECTED

BULLETIN WIND ADVISORY UNTIL 8 PM

2 · App documentation

Web App Development for Mobile — Midterm. The app spec I gave Claude, plus how the finished app works.

Also included: **PROMPTS.md** (every prompt I sent Claude) and **API.md** (the API reference).

1. The brief I gave Claude

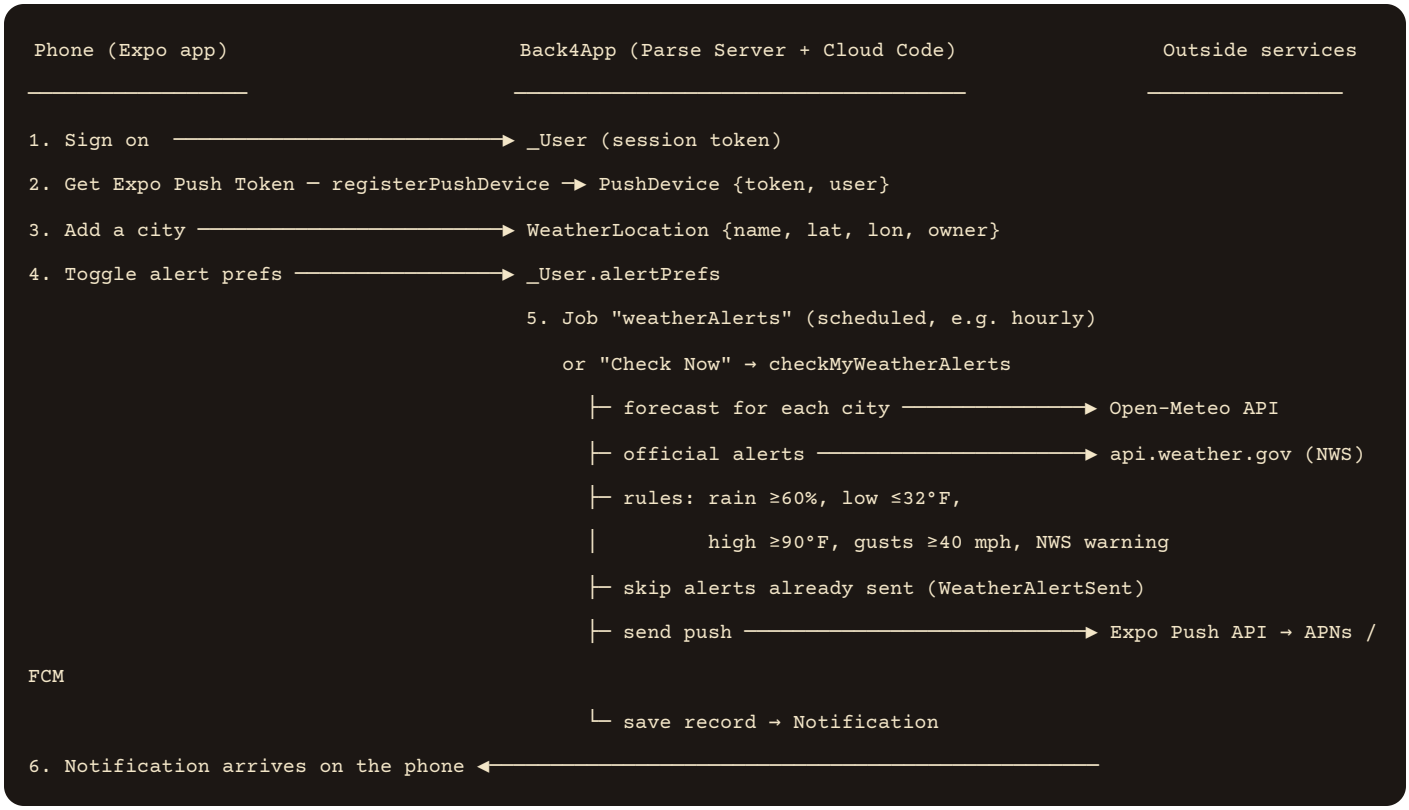
> Build a **full Expo app** that is a **weather app** with a **unique theme** designed in **Claude Design** (not the default weather look). It must have **logins and sign-ups**, and **push notifications for weather alerts sent from the backend using Back4App**. Reuse the Back4App app, Expo account and push setup from my earlier project.

Theme chosen: *Retro radio / weather-station dashboard.* A 1940s tabletop radio with a walnut cabinet, cream dial face and brass trim. The temperature is an analog gauge with a needle, saved cities are radio "stations" and "presets", and alerts arrive as an "Emergency Broadcast" on red ticker tape.

2. Features

Screen	What it does
Sign On	Log in, or create an account ("New Listener") with a username, password and optional email. Accounts are stored in Back4App's `_User` table.
Now	Analog temperature gauge, current conditions, feels-like / humidity / wind readouts, a bulletin bar (official NWS alert or rain warning), and a scrollable 12-hour "tuning band". Pull down to refresh.
7-Day	Daily forecast: conditions, rain chance, wind gusts, high/low, plus sunrise/sunset and UV index.
Stations	Search any city (Open-Meteo geocoding), add it as a preset, or **Use my location** (GPS). Tap to tune in; long-press to remove. Saved per user in Back4App.
Alerts	Inbox of every notification received on the phone (tap one, or tap a banner, to read the full message). Toggles for which alerts you want (master switch, rain, freezing, heat, wind, official NWS warnings), saved to your Back4App user. **Check Now** runs the backend alert check, and **Test Alert** sends a sample push. Also shows a broadcast log of every notification sent, and lets you sign off.

3. How push weather alerts work (backend)



Back4App classes

Class	Fields	Who can read it
`_User`	username, password (hashed), email, `alertPrefs`	the user
`WeatherLocation`	name, region, country, latitude, longitude, owner	owner only (ACL)
`PushDevice`	token, platform, deviceName, user	Cloud Code only
`WeatherAlertSent`	user, location, key (for example `rain:2026-10-22`)	Cloud Code only
`Notification`	title, message, sender, target, devices, sent, status	sender; admins via Cloud Code

Cloud Code (`cloud/main.js`)

Function / job	Purpose
`registerPushDevice`, `unregisterPushDevice`	Save or remove this phone's Expo Push Token
`checkMyWeatherAlerts({ force })`	Check the logged-in user's cities now and push any alerts
`sendTestWeatherAlert`	Push the current temperature for the first city (demo)
job `weatherAlerts`	Scheduled check for **every** user, which sends the alerts automatically
`sendPushNotification`, `broadcastPushNotification`	Manual sends (used by `push.html`, the admin web sender)

Function / job	Purpose
`getPushReceipts`	Checks with the Expo Push Receipts API that Apple/Google delivered a notification
`getNotificationHistory`	Broadcast log

4. APIs used

- **Open-Meteo** forecast and geocoding: free, no API key
- **National Weather Service** active alerts (`api.weather.gov`): free, no key, US locations only
- **Back4App REST API**: auth, data and Cloud Functions
- **Expo Push API**: delivery to iPhone (APNs) and Android (FCM)
- **Expo Push Receipts API**: confirms Apple/Google actually delivered each notification

5. Tech

Expo SDK 57 (React Native 0.86), `expo-notifications`, `expo-location`, `expo-font` with the Google Fonts Righteous, Barlow Condensed and Courier Prime, and `react-native-svg` for the gauge. Back4App is called through its REST API with `fetch`, because the Parse JS SDK doesn't bundle in Expo Go.

6. Design

Made in **Claude Design**. The files are in `design/`: `Main`, `Forecast`, `Stations`, `Alerts`, `SignOn` and `StyleGuide` (`.dc.html`) plus `canvas.json`.

Token	Hex	Use
Walnut	`#3b2416`	cabinet background
Bakelite	`#1c1714`	readouts, nav
Dial cream	`#f3e6c8`	panels
Brass	`#c9a24a`	trim, selected tab
Tube amber	`#f2a93b`	readout numbers
Signal red	`#c8402a`	alerts, main buttons
Cold steel	`#2f5d73`	lows, rain

3 · API reference

WX Radio uses **3 public weather APIs**, **the Back4App API** (including our own Cloud Function API) and **the Expo Push API**.

#	API	Used for	Auth	Code
1	Open-Meteo Forecast	Current, hourly, 7-day weather	none (free)	`src/weather.js` → `getForecast()`; `cloud/main.js` → `fetchForecast()`
2	Open-Meteo Geocoding	City search	none (free)	`src/weather.js` → `searchCities()`
3	National Weather Service (NWS)	Official alerts/warnings (US)	none (needs a User-Agent)	`src/weather.js` → `getOfficialAlerts()`; `cloud/main.js` → `fetchOfficialAlerts()`
4	Back4App (Parse) REST API	Sign up, login, saved cities, alert prefs	App ID + JavaScript Key (+ session token)	`src/back4app.js`
5	Back4App Cloud Functions (our API)	Push devices, weather alerts, test alert, history	same	`cloud/main.js`
6	Expo Push API	Deliver notifications to iPhone/Android	none (Expo Push Token)	`cloud/main.js` → `deliver()`
7	Expo Push Receipts API	Confirm Apple (APNs) / Google (FCM) delivered each notification	none (ticket ids)	`cloud/main.js` → `fetchReceipts()`, `getPushReceipts`

1. Open-Meteo Forecast API

`GET https://api.open-meteo.com/v1/forecast`

Example request (Newark, NJ):

```
https://api.open-meteo.com/v1/forecast?latitude=40.7357&longitude=-74.1724

&current=temperature_2m,apparent_temperature,relative_humidity_2m,weather_code,wind_speed_10m,wind_gusts_10m,is_day,precipitation
  &hourly=temperature_2m,precipitation_probability,weather_code,wind_gusts_10m,is_day

&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max,wind_gusts_10m_max,sunrise,sunset,uv_index_max
  &temperature_unit=fahrenheit&wind_speed_unit=mph&timezone=auto&forecast_days=7
```

Response (shortened):

```
{
  "current": { "time": "2026-10-22T13:15", "temperature_2m": 54.2, "apparent_temperature": 51,
               "relative_humidity_2m": 68, "weather_code": 2, "wind_speed_10m": 12 },
  "hourly":  { "time": ["2026-10-22T13:00", "..."], "temperature_2m": [54, "..."], "precipitation_probability":
[10, "..."] },
  "daily":   { "time": ["2026-10-22", "..."], "temperature_2m_max": [61, "..."], "temperature_2m_min": [47,
"..."] }
}
```

`weather_code` is a WMO code, which `describe()` in `src/weather.js` turns into "Partly cloudy", "Rain" and so on.

2. Open-Meteo Geocoding API

`GET https://geocoding-api.open-meteo.com/v1/search?name=Newark&count=8&language=en&format=json`

```
{ "results": [ { "id": 5101798, "name": "Newark", "admin1": "New Jersey", "country_code": "US",
                 "latitude": 40.73566, "longitude": -74.17237, "timezone": "America/New_York" } ] }
```

3. National Weather Service — active alerts

`GET https://api.weather.gov/alerts/active?point=40.7357,-74.1724` Headers: `User-Agent: WeatherAlertsMidterm (github.com/AjHy2003)`, `Accept: application/geo+json`

```
{ "features": [ { "properties": {
    "id": "urn:oid:2.49.0.1.840.0....", "event": "Wind Advisory", "severity": "Moderate",
    "headline": "Wind Advisory issued October 22 at 9:15AM EDT until October 22 at 8:00PM EDT" } } ] }
```

Alerts with severity **Moderate, Severe or Extreme** become push notifications.

4. Back4App REST API

Base URL `https://parseapi.back4app.com`. Every request sends:

```
X-Parse-Application-Id: <Application ID>
X-Parse-Javascript-Key: <JavaScript Key>
X-Parse-Session-Token: <token>     (after login)
Content-Type: application/json
```

Action	Request
Sign up	`POST /users` `{"username","password","email","alertPrefs"}` → `{"objectId","sessionToken"}`
Log in	`POST /login` `{"username","password"}` → user + `sessionToken`
Current user	`GET /users/me`
Log out	`POST /logout`
Save alert settings	`PUT /users/<objectId>` `{"alertPrefs":{"enabled":true,"rain":true,...}}`
List my cities	`GET /classes/WeatherLocation?where={"owner":{"__type":"Pointer","className":"_User","objectId":"<id>"}}`
Add a city	`POST /classes/WeatherLocation` `{"name","region","latitude","longitude","owner","ACL"}`
Remove a city	`DELETE /classes/WeatherLocation/<objectId>`

5. Our API — Back4App Cloud Functions (`cloud/main.js`)

Called with `POST https://parseapi.back4app.com/functions/<name>`, using the same headers.

Function	Body	Returns
`registerPushDevice`	`{"token":"ExponentPushToken[...]","platform":"ios","deviceName":"iPhone"}`	`{"objectId"}`
`unregisterPushDevice`	`{"token"}`	`{"removed":1}`
`checkMyWeatherAlerts`	`{"force":false}`	`{"locations":1,"found":[{"title":"🌧️ Rain likely in Newark",...}],"sent":1,"skipped":0}`
`sendTestWeatherAlert`	`{}`	`{"sent":1,"failed":[],"notificationId":"..."}`
`getPushReceipts`	`{"notificationId":"..."}`	`{"receiptStatus":"delivered","delivered":1,"pending":0,"errors":[]}`
`getNotificationHistory`	`{"limit":20}`	`[{"title","message","status","createdAt",...}]`
`sendPushNotification`	`{"title","message"}`	`{"sent","failed"}` (own devices)
`broadcastPushNotification` *(admin)*	`{"title","message","username?"}`	`{"sent","failed"}` (everyone or one user)
Job `weatherAlerts`	(scheduled hourly)	checks every user's cities and sends alerts

Try it from PowerShell (after logging in, use your session token):

```
$h = @{ "X-Parse-Application-Id"="<APP_ID>"; "X-Parse-Javascript-Key"="<JS_KEY>"; "X-Parse-Session-Token"="<SESSION>"; "Content-Type"="application/json" }
Invoke-RestMethod -Method Post https://parseapi.back4app.com/functions/sendTestWeatherAlert -Headers $h -Body "{}"
```

6. Expo Push API

`POST https://exp.host/--/api/v2/push/send` (sent by Cloud Code, never by the phone)

```
[ { "to": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]", "title": "🌧️ Rain likely in Newark",
    "subtitle": "WX Radio · Newark", "priority": "high", "channelId": "default",
    "body": "85% chance of rain around 4 PM. Grab an umbrella!", "sound": "default",
    "data": { "type": "weather", "alert": "rain" } } ]
```

Response: `{ "data": [ { "status": "ok", "id": "..." } ] }`. Tokens that come back as `DeviceNotRegistered` are deleted from `PushDevice`.

7. Expo Push Receipts API

`POST https://exp.host/--/api/v2/push/getReceipts` (from Cloud Code, a few seconds after sending)

The send call above only says Expo **accepted** the message. The receipt says whether **Apple (APNs) / Google (FCM) delivered** it.

```
{ "ids": ["XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"] }
```

Response:

```
{ "data": { "XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX": { "status": "ok" } } }
```

or, on failure:

```
{ "data": { "XXXXXXXX-...": { "status": "error", "message": "...", "details": { "error": "DeviceNotRegistered" } } } }
```

Our Cloud Function `getPushReceipts` saves the result on the `Notification` row (`receiptStatus`: delivered / error / pending, plus `receiptErrors`). The app's **Test Alert** button and `wx-broadcast.html` check it automatically about 5 seconds after sending.

Receipt error	Meaning / fix
`DeviceNotRegistered`	The app was removed or the token changed. Sign off and sign back on.
`InvalidCredentials`	APNs/FCM keys in EAS are missing or wrong. Run `npx eas-cli@latest credentials`.
`MessageTooBig`	The payload is over 4 KB.
`MessageRateExceeded`	Too many messages to one device. Slow down.

4 · Prompt log

All the prompts I sent Claude (Claude Code) while building this project, in order, copied as I typed them. Screenshots and pasted errors are described in *[brackets]*. **Keys, passwords and private files are redacted.**

- **Part 1** is the midterm weather app (WX Radio).
- **Part 2** is the earlier session where we built the Expo + Back4App login and push-notification setup that WX Radio reuses.

Part 1 — Midterm: WX Radio weather app

#	My prompt	What Claude did
1	for our midterm we have to make an app with api the push notifications and so on wanna do the yu gi deck or pokemon deck app if thats okay	Suggested keeping the Yu-Gi-Oh! app and asked for the rubric
2	hopefully we dont have to make a whole new one i was one of the only students to get this working	Listed what was already done and asked for the assignment details
3	*[Screenshot of the Canvas midterm assignment: video of the app working, app documentation given to Claude, design files, Back4App screenshots of user logins and push notifications; weather app made with Claude Design and a unique theme; must use logins and sign-ups; must use push notifications for weather alerts from the backend using Back4App; must be a full Expo app]*	Explained the midterm has to be a weather app. Planned to reuse the Back4App login and push backend, and asked which theme
4	let's do this when i come back since it's due tomorrow	Listed what to have ready and offered to start on the backend
5	okay lets do it	Built the weather backend (Cloud Code alert rules, scheduled job) and asked for the theme
6	*Theme picked:* **Retro radio / dashboard**	Made the design in **Claude Design** (6 screens + style guide), then built the Expo app to match: login/sign-up, Now gauge, 7-Day, Stations, Alerts
7	I need apis as well	Wrote `API.md` documenting every API (Open-Meteo, NWS, Back4App REST, our Cloud Functions, Expo Push)
8	we need the test notifications should and the html that i can do random notifications should i do another app build and build with ai or no	Confirmed no new build was needed. Made `wx-broadcast.html` (weather presets + random alert generator)
9	when doing the test notifiactions in the app It said invalid test notifications	Explained that the new Cloud Code wasn't deployed yet and gave the steps
10	whats the stuff i have to paste	Sent the full `cloud/main.js` file to paste into Back4App
11	I think i need what prompts I sent you along with the documentation	Wrote this prompt log

Part 2 — Earlier session: Expo + Back4App + push notifications setup

#	My prompt	What Claude did
1	Can you make me a standard expo app that uses back4end expo go dev you can even code it on github make it make it a yugioh deck builder app that optimizes the best deck building or if its easier you can use an anime generator app that allows me to generate shows based off my interest whatever is easier you can use clis if you want	Created the Expo app (Yu-Gi-Oh! deck optimizer) with Back4App storage and pushed it to GitHub
2	does this have notifications and what not	Explained the notification options
3	*[Screenshot of the class slide: 1) add expo-notifications and get the Expo Push Token, 2) save the token in a Back4App* `PushDevice` *class, 3) create a Cloud Code function that sends to the Expo Push API, 4) set up Apple/Android credentials in Expo/EAS, 5) test on a real phone]*	Built all of it: login, `PushDevice`, the `sendPushNotification` Cloud Function, EAS config
4	whats left	Listed the remaining setup steps
5	*[Back4App keys — redacted]*	Added support for the JavaScript Key
6	*[PowerShell error: cannot find path* `yugioh-deck-builder` *]*	Showed how to clone the repo on Windows
7	what about step 2 3 4 and 5	Gave step-by-step Back4App, EAS, Firebase and build instructions
8	what code to open cloud main.js	Gave the PowerShell commands to open or copy the file
9	*[Pasted the Cloud Code to confirm]*	Confirmed and gave the deploy steps
10	it deployed whats next	Explained the EAS login and init
11	*[Output showing the EAS projectId]*	Explained Firebase setup for Android
12	*[Screenshot of the Firebase console]*	Walked through adding the Android app
13	i have both files	Explained uploading the FCM V1 key to EAS
14	does it also have apple	Explained that iPhone push works in Expo Go
15	*[Screenshot: EAS credentials menu on FCM Legacy]*	Pointed to the correct FCM V1 option
16	*[Screenshot: EAS asking for the key file path]*	Explained how to copy the file path
17	*[Uploaded the Firebase service-account key — redacted]*	Gave the path and recommended rotating the key
18	its fine keep going i got it	Gave the Android build steps
19	okay now iphone	Gave the Expo Go iPhone steps
20	did you create a login and everything	Described the login features
21	*[Error: ENOENT ... .expo\codesigning ...]*	Gave a fix for the Windows Expo bug
22	*[Error: Remove-Item cannot find path ...]*	Had me create the missing folder
23	*[PowerShell output: folder created]*	Next steps

#	My prompt	What Claude did
24	need to be logged into my expo account	Showed how to stay logged in (offline mode workaround)
25	is it supposed to be launched in browser	Explained it runs in Expo Go on the phone
26	its running 10.62 on chrome	Explained the tunnel for campus Wi-Fi
27	*[Screenshot: Expo Go "signed in to Expo Go but not Expo CLI"]*	Showed how to log in to the Expo CLI
28	you have to use tunnel	Gave the tunnel commands and added an `npm run tunnel` script
29	*[Screenshot of the app login screen]* Logged in but can't login	Explained the app account is separate from the Expo account
30	It said error unauthorized	Made the app send the key under every header name
31	*[.env contents — redacted]*	Troubleshooting
32	it only allows me to click log in which gives me unathourized	Gave a PowerShell key test
33	*[Test output: 401 Unauthorized]*	More troubleshooting
34	still same issue	Asked for the error body
35	Status: 401 Body: {"error":"unauthorized"}	Pointed to where the keys are in the dashboard
36	*[Screenshot: Back4App API console]*	Explained where to find the keys
37	its greyed out	Pointed to the other place
38	*[Screenshot: Back4App Connect page]*	Pointed to App Settings → Security & Keys
39	*[Screenshot: Security & Keys — redacted]*	Found the problem: the Client Key had been used as the Application ID
40	just paste it for me and give me everything	Wrote a script that found the right App ID and saved `.env`
41	*[Screenshot: logged in, "Invalid function: registerPushDevice"]*	Explained the Cloud Code wasn't deployed to that app
42	*[Pasted Back4App's default Cloud Code — contained a secret key, redacted]*	Explained what to replace it with
43	*[Pasted the Cloud Code into PowerShell by mistake]*	Explained it goes in the browser
44	what code	Pasted the full Cloud Code
45	*[Screenshot: "Parse.Cloud.httpRequest is not a function"]*	Fixed the Cloud Code to use `fetch` (newer Parse Server)
46	how about android	Gave the Android development build steps
47	I wanna make a webpage that i can type in that can send to the app	Built `push.html` and the admin broadcast Cloud Function

#	My prompt	What Claude did
48	*[Screenshot: push.html keys screen]*	Explained what to fill in
49	worked now last thing can you have every notifications sent saved on the bac4end app	Saved every send to the `Notification` class, with history on the page
50	*[ConfigError: package.json does not exist]*	Explained the correct folder
51	okay what should i paste in main.js	Pasted the latest Cloud Code
52	*[Same ConfigError again]*	Gave a single command that works from any folder
53	make a zip of this chat so i can import it into another chat please	Made a handoff zip with a summary and the code